eURC

Point of Sale on Azure

Hands-On Deployment Guide

Hands-on Project 02 — Dev, Test and Production with DevOps CI/CD (GitHub Actions)

Document	EURC-P02-HOD · Hands-On Deployment Guide
Version	1.0 · 23 October 2026
Author	M. Akharuzzaman
Application	eURC POS — React 19 front end, Node.js 22 / Express 5 API
Platform	Microsoft Azure (Canada Central) · GitHub Actions
Team	2 developers (Developer 1 = lead) · 1 tester (QA engineer)
Kit	eurc-p02-cicd-kit.zip
Companion documents	Hands-on Project 02 Report · Application, DevOps and Azure architecture drawings (A3) · docs/RUNBOOK.md · docs/TEAM.md

NOTE This guide is written to be followed in order. Every phase ends with an **expected result** and a box to record what you saw. Keep the completed guide as evidence for your lab submission.

About this guide

Document control

Version	Date	Author	Change
1.0	2026-10-23	M. Akharuzzaman	First issue for the first live deployment

Conventions

Marker	Meaning
Dark box	Commands to type or paste into Azure Cloud Shell (Bash). Grey text after # is a comment and does nothing.
EXPECTED RESULT	What you should see when a step has worked. Do not continue until it matches.
CAUTION / STOP	A decision point or something that costs money or cannot be undone.
WHO	Which team member performs the phase: Lead (Developer 1), Developer 2 or Tester.
RECORD YOUR RESULT	Fill in date, time and outcome. This becomes your evidence.
<placeholder>	Replace with your own value, without the angle brackets.

Contents

1. Overview	4
Phase 0 — Prerequisites and team	6
Phase 1 — Prepare Azure Cloud Shell	7
Phase 2 — Configure the lab	8
Phase 3 — Sign in to GitHub from Cloud Shell	9
Phase 4 — Preflight checks	10
Phase 5 — Bootstrap Azure	11
Phase 6 — Set up GitHub and push	12
Phase 7 — First pipeline run: build, dev, test	13
Phase 8 — QA sign-off and production release	14
Phase 9 — Verify the deployment	16
Phase 10 — Harden production and protect main	17
Phase 11 — Monitor	18
Phase 12 — Team exercises	19
Phase 13 — Collect evidence	22
Phase 14 — Tear down	23
Appendix A — Troubleshooting	24
Appendix B — Resource names	25
Appendix C — Lab completion sign-off	26

1. Overview

By the end of this lab you will have a multi-store Point of Sale running in three Azure environments, released by a GitHub Actions pipeline that builds the application once and promotes the same package from dev to test to production, with automated quality gates, the tester's QA sign-off, a zero-downtime slot swap and automatic rollback.

1.1 What gets deployed

	dev	test	prod
Resource group	rg-eurc-dev-cc	rg-eurc-test-cc	rg-eurc-prod-cc
App Service plan	Linux B1	Linux B1	Linux S1 + staging slot
Data	Azure SQL Basic, Blob, Key Vault (Entra ID only)	same as dev	same, behind private endpoints
Released	automatically on each merge	automatically after dev	after the tester approves
Quality gates	smoke	smoke, integration, load, Playwright E2E	staging smoke, verify, post-deploy gate

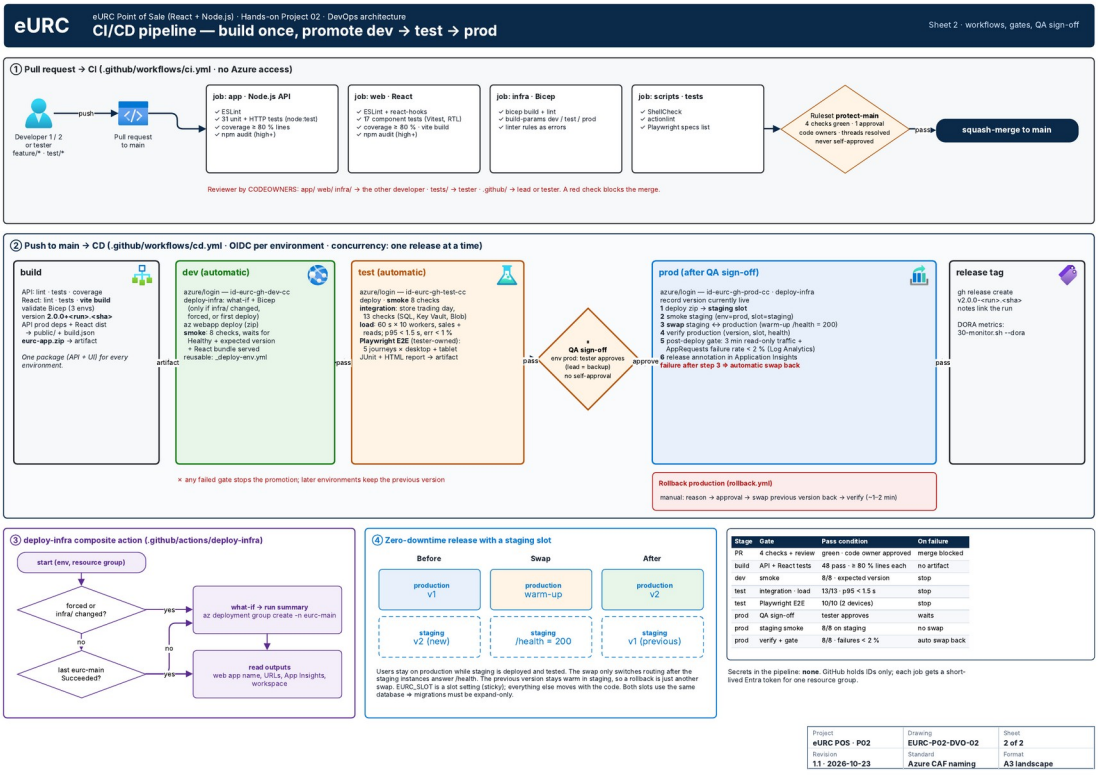


Figure 1 — The CI/CD pipeline (drawing EURC-P02-DVO-02)

1.2 Phases and timeline

Phase	Activity	Who	Time
0	Accounts and team ready	Everyone	10 min
1	Prepare Cloud Shell and upload the kit	Lead	5 min
2	Configure config/eurc.env	Lead	5 min
3	Sign in to GitHub from Cloud Shell	Lead	3 min
4	Preflight checks	Lead	3 min
5	Bootstrap Azure (resource groups, OIDC identities, budget)	Lead	5 min
6	Set up GitHub and push (starts the first pipeline run)	Lead · team accepts invitations	5 min
7	First pipeline run: build, dev, test	Watch (everyone)	25–35 min
8	QA sign-off and production release	Tester	10–15 min
9	Verify the deployment and use the POS	Tester · Lead	20 min
10	Harden production and protect main	Lead	5 min
11	Monitor	Lead	10 min
12	Team exercises (6)	Whole team	60–90 min
13	Collect evidence	Everyone	15 min
14	Tear down	Lead	20 min

> **CAUTION** Running cost while everything is up is about **USD 4.90 per day** (mainly the S1 plan in production). Plan to finish the lab within one or two days and always run Phase 14.

Phase 0 — Prerequisites and team

Goal	Everyone has the accounts and access the lab needs.
Who	Everyone
Time	10 minutes
Changes	Nothing

0.1 Accounts

Item	Required for	Check
Azure for Students subscription, role Owner	Lead	☐
GitHub account for each team member	Lead, Developer 2, Tester	☐
A public GitHub repository is acceptable (needed on GitHub Free for reviews and production approval); for a private one, GitHub Pro from the Student Developer Pack	Lead	☐
E-mail address that will receive alerts and budget warnings	Lead	☐
Project 01 (eURC single environment) torn down, to save credit	Lead	☐
The kit eurc-p02-cicd-kit.zip downloaded to your computer	Lead	☐

0.2 Team roles

Role	Setting in config/eurc.env	Responsibilities in this lab
Lead (Developer 1)	GITHUB_OWNER and DEV1	Runs the scripts, owns the repository and Azure, reviews Developer 2, backup production approver
Developer 2	DEV2	Builds features, reviews the lead's pull requests
Tester (QA engineer)	TESTER	Owns tests/, explores releases on test, approves production (QA sign-off)

> **NOTE** Working alone? Set `DEV2=""` and `TESTER=""`. You then approve production yourself. Skip `--protect-main` in Phase 10, because it requires a review by someone other than the author.

RECORD YOUR RESULT

Team (Lead / Developer 2 / Tester)	
Alert e-mail · date started	

Phase 1 — Prepare Azure Cloud Shell

Goal	Open a Bash session in Azure and unpack the kit.
Who	Lead
Time	5 minutes
Changes	Files in your Cloud Shell storage only

1. Open **https://portal.azure.com** and sign in with your Azure for Students account.
2. Click the Cloud Shell icon (>_) in the top bar and choose **Bash**. Accept the storage prompt the first time.
3. Click **Manage files** → **Upload** and select `eurc-p02-cicd-kit.zip`.
4. Start a tmux session (keeps long commands running if the browser disconnects) and unpack the kit:

```
tmux new -s eurc
unzip -o eurc-p02-cicd-kit.zip && cd eurc-platform
chmod +x setup.sh scripts/*.sh .github/scripts/*.sh
ls
```

NOTE If Cloud Shell disconnects later, reopen it and run `tmux attach -t eurc` to return to where you were.

EXPECTED RESULT The listing shows `app config docs infra scripts setup.sh tests` web and README.md.

RECORD YOUR RESULT

Cloud Shell opened (date / time)	
Kit unpacked (yes / no)	

Phase 2 — Configure the lab

Goal	Tell the scripts who the team is and where to send alerts.
Who	Lead
Time	5 minutes
Changes	config/eurc.env only

1. Open the settings file:

```
nano config/eurc.env
```

2. Change the lines marked CHANGE ME (GitHub user names are case-sensitive):

```
GITHUB_OWNER="<lead-github-user>"        # owns the repository
DEV1="<lead-github-user>"                # usually the lead
DEV2="<developer-2-github-user>"         # or "" if working alone
TESTER="<tester-github-user>"            # or "" if working alone
ALERT_EMAIL="<your-email>"
```

3. Save with **Ctrl+O**, **Enter**, then **Ctrl+X**.
4. Optional settings you can leave as they are:

Setting	Default	Purpose
GITHUB_REPO	eurc-platform	Repository name created in Phase 6
GITHUB_VISIBILITY	public	public works on GitHub Free; private needs GitHub Pro / Team
BUDGET_AMOUNT	60	Monthly budget (subscription currency) with e-mail alerts
LOCATION / LOC_SHORT	canadacentral / cc	Azure region and its short code in resource names

EXPECTED RESULT Check the file: `grep -E "OWNER|DEV|TESTER|EMAIL" config/eurc.env` shows your values and no CHANGE-ME text.

RECORD YOUR RESULT

Values entered and checked	

Phase 3 — Sign in to GitHub from Cloud Shell

Goal	Allow the scripts to create the repository, environments and team settings.
Who	Lead
Time	3 minutes
Changes	A GitHub token stored in your Cloud Shell

1. Run the GitHub CLI login with the workflow scope (needed to push pipeline files):

```
gh auth login -s workflow
```

2. Answer: **GitHub.com → HTTPS → Yes (authenticate Git) → Login with a web browser**.
3. Copy the one-time code, open the link it prints in a new browser tab, paste the code and approve.
4. Confirm:

```
gh auth status
```

> **EXPECTED RESULT** Logged in to github.com as `<lead-github-user>`, with token scopes including `repo` and `workflow`.

RECORD YOUR RESULT

GitHub user shown by gh auth status	

Phase 4 — Preflight checks

Goal	Confirm the subscription can host the lab before anything is created.
Who	Lead
Time	3 minutes
Changes	Registers missing Azure resource providers only

```
./scripts/00-preflight.sh
```

The script checks, in order:

- tools (az, gh, git, jq, node, curl, zip) and the values in config/eurc.env
- your Azure login and that you have the Owner role on the subscription
- resource providers (Web, Sql, Storage, KeyVault, Insights, Network, …)
- that Canada Central is allowed by the subscription policy
- that Azure SQL Basic and App Service B1 and S1 (Linux) are offered to your subscription
- your GitHub login

EXPECTED RESULT Every line shows [OK] and the last line reads `Preflight passed. Next: ./scripts/01-bootstrap-azure.sh`.

Decision: S1 not available

Some student subscriptions have no Standard (S1) quota. If preflight stops with "App Service S1 (Linux) not offered", switch production to B1 without a staging slot. The pipeline then deploys straight to production (a brief restart) instead of swapping, and the rollback swap is not available.

```
nano infra/env/prod.bicepparam
#   change:  param appServiceSku = 'B1'
#   change:  param enableStagingSlot = false
./scripts/00-preflight.sh   # now warns about S1 and passes
```

CAUTION Only change this if preflight told you to. In 20-test-env.sh the two prod checks "staging slot exists" and "plan tier Standard" will then report as expected failures.

RECORD YOUR RESULT

Preflight result (passed / issue)	
S1 available (yes / no)	

Phase 5 — Bootstrap Azure

Goal	Create what the pipeline cannot create for itself.
Who	Lead
Time	5 minutes
Changes	4 resource groups, 3 managed identities with federated credentials, 6 role assignments, 1 budget

```
./scripts/01-bootstrap-azure.sh
```

Created	Name	Purpose
Resource groups	rg-eurc-cicd-cc, rg-eurc-dev-cc, rg-eurc-test-cc, rg-eurc-prod-cc	One per environment plus one for deployment identities
Deployment identities	id-eurc-gh-dev-cc, id-eurc-gh-test-cc, id-eurc-gh-prod-cc	GitHub Actions signs in as these with OpenID Connect — no secrets
Federated credentials	repo:<owner>/eurc-platform:environment:<env>	Each identity trusts only jobs in the matching GitHub environment
Role assignments	Contributor + constrained RBAC Administrator	Each identity can change only its own resource group
Budget	budget-eurc-monthly	E-mails at 50 % and 80 % actual, 100 % forecast

EXPECTED RESULT The script ends with "Bootstrap complete" and prints tenant, subscription and three client IDs. These are identifiers, not secrets. A budget warning ("some student offers do not support budgets") is acceptable.

Verify in the portal:

- Resource groups
- rg-eurc-cicd-cc → each identity → Federated credentials

```
az group list --query "[?starts_with(name,'rg-eurc')].name" -o table
```

RECORD YOUR RESULT

Resource groups created (4)	
Budget created (yes / skipped)	

Phase 6 — Set up GitHub and push

Goal	Create the repository and team rules, then push the kit. The push starts the first pipeline run.
Who	Lead · team accepts invitations
Time	5 minutes
Changes	GitHub repository, collaborators, CODEOWNERS, environments, variables, first commit

```
./scripts/02-setup-github.sh
```

Step in the script	Result
Repository	<owner>/eurc-platform created (public by default)
Team: collaborators	Developer 2 and Tester invited with write access
CODEOWNERS	app/, web/, infra/, scripts/ → developers · tests/ → tester · .github/ → lead + tester
Environments	dev and test: main branch only · prod: main only, approval by the tester (lead as backup), self-approval blocked
Variables	AZURE_TENANT_ID, AZURE_SUBSCRIPTION_ID, and per environment AZURE_CLIENT_ID, AZURE_RESOURCE_GROUP, ALERT_EMAIL (prod)
Push	First commit to main → CD pipeline starts

WHO Developer 2 and Tester: open the GitHub e-mail invitation and click **Accept invitation** now.

CAUTION If the script printed "tester could not be added as reviewer yet", the tester had not accepted in time. After the tester accepts, the lead runs `./scripts/02-setup-github.sh` once more. It is safe to repeat.

EXPECTED RESULT "Pushed. Watch it: https://github.com/<owner>/eurc-platform/actions". In GitHub, Settings → Environments lists dev, test and prod.

RECORD YOUR RESULT

Repository URL	
Invitations accepted (Dev 2 / Tester)	
Pipeline run number	

Phase 7 — First pipeline run: build, dev, test

Goal	Watch the pipeline create the environments and pass every automated gate.
Who	Everyone (watch)
Time	25–35 minutes (first run creates all infrastructure)
Changes	All Azure resources in dev and test; prod resources are created in Phase 8

1. Open `https://github.com/<owner>/eurc-platform/actions` and click the running **CD** workflow.
2. Follow the jobs in the graph. Click a job to see live logs; the run Summary page shows the what-if change list and a test table for each stage.

Job	What happens	Typical time	Passes when
build	API tests (31), React tests (17), coverage, React build, Bicep validation, one zip package	3–4 min	all tests and coverage gates green
dev	First run: Bicep creates dev (≈ 8 min). Deploys the zip. Smoke suite.	10–12 min	smoke 8 / 8, expected version
test	Creates test. Deploys. Smoke, integration (13), load (60 s) and Playwright (10 browser tests)	14–18 min	all four suites green
prod	Waits for review (orange clock icon)	—	Phase 8

EXPECTED RESULT When the test job is green, open its URL (shown on the job box). The POS sign-in page shows an amber **TEST** badge with the version, for example `test · production · v2.0.0+1.abc1234`.

CAUTION If a job fails, open it, read the first red step and check Appendix A. After a fix, use "Re-run failed jobs". Do not delete resource groups.

RECORD YOUR RESULT

build result / duration	
dev result / URL	
test result / URL	
Playwright report downloaded (reports-test artifact)	

Phase 8 — QA sign-off and production release

Goal	The tester verifies the release on test and approves production.
Who	Tester (Lead if working alone)
Time	10–15 minutes
Changes	Production resources, staging slot, alerts; first live release

8.1 Tester checks the release on test

1. Open the test URL. Choose Create a store, enter a store name, your name, a store code (e.g. qa-check), e-mail and a password of at least 10 characters with letters and digits.
2. Products → add two products (for example COF-001 Coffee 2.45, 13 % tax, stock 20; BAK-002 Muffin 2.95, 0 % tax, stock 5).
3. Register → add 2 × Coffee and 1 × Muffin. Total should read $8.49. Charge → Cash → $10.00 → change $1.51 → Complete sale.
4. Report → the day shows 1 transaction, gross $8.49.
5. In the pipeline run, open Artifacts → reports-test → e2e-html/index.html to review the Playwright results.

8.2 Approve production

1. In the CD run, click **Review deployments**, tick **prod**, add a comment such as "QA passed: smoke, E2E, manual checkout" and click **Approve and deploy**.
2. Watch the prod job:

Step	What happens
Infrastructure	First run: creates VNet, private endpoints, DNS zones, S1 plan, staging slot, alerts (≈ 12–15 min)
Deploy to staging slot	New version goes to the staging slot; users stay on production
Smoke test staging	8 checks against the staging URL (environment prod, slot staging)
Swap	Staging and production swap after /health returns 200 — no downtime
Verify production	Smoke suite against the production URL
Post-deployment gate	3 minutes of read-only traffic, then failure rate of this version from Log Analytics (< 2 %)
release tag	GitHub release v2.0.0-<run>.<sha> is created

EXPECTED RESULT All jobs green. The production URL shows a blue **PROD** badge with the same version as test. An e-mail "slot swap" alert arrives at ALERT_EMAIL.

> **NOTE** On the very first release there is no previous version in the staging slot, so an automatic rollback is skipped if the gate fails. From the second release on, a failed gate swaps the previous version back automatically.

RECORD YOUR RESULT

Approved by / time	
Production URL	
Version in production	
Release tag	

Phase 9 — Verify the deployment

Goal	Prove each environment matches its design and the POS works end to end.
Who	Lead (scripts) · Tester (POS walkthrough)
Time	20 minutes
Changes	Test stores in each environment (deleted by the integration suite)

9.1 Conformance and functional tests

```
./scripts/20-test-env.sh --integration
```

Environment	Checks	Includes
dev	21 + integration	HTTPS only, TLS 1.2, FTP off, managed identity, SQL Entra-only, no shared keys, Key Vault RBAC, no slot, smoke
test	21 + integration	same as dev
prod	29 + integration	plus S1 tier, staging slot, VNet integration, 3 approved private endpoints, alerts, availability test, Key Vault and Storage refuse internet access (403)

EXPECTED RESULT Each environment ends with "[OK] <env>: N checks passed" and a report file in reports/.

9.2 Use the POS in production

1. Open the production URL → Create a store (your real store name).
2. Products → add three or four products with stock.
3. Staff → add a cashier with a temporary password.
4. Sign out → sign in as the cashier → ring up a cash sale and a card sale (enter any approval code such as AUTH-1234).
5. Print the receipt (browser print dialog) → save as PDF for your evidence.
6. Sign in as manager → Sales → open a receipt → Refund sale → Report shows the refund.

RECORD YOUR RESULT

20-test-env.sh totals (dev / test / prod)	
Production walkthrough completed	
Receipt PDF saved	

Phase 10 — Harden production and protect main

Goal	Lock the production data and turn on the team review rules.
Who	Lead
Time	5 minutes
Changes	Delete locks on prod SQL, Storage, Key Vault; branch ruleset

```
./scripts/03-harden-prod.sh
./scripts/02-setup-github.sh --protect-main   # skip if working alone
```

EXPECTED RESULT 03 shows three "CanNotDelete" locks, the private endpoints "Approved", and public network access "Disabled". 02 shows "main now requires a pull request, 1 approval (code owners for their area) …".

From now on nobody pushes directly to main. Every change goes through a pull request with the four green checks (app, web, infra, scripts) and an approval by a code owner who is not the author.

RECORD YOUR RESULT

Locks created (3)	
Ruleset protect-main active	

Phase 11 — Monitor

Goal	See the platform the way an operator does.
Who	Lead
Time	10 minutes
Changes	Test traffic on test; one test e-mail

```
./scripts/30-monitor.sh                 # versions, health, traffic, alerts, cost
./scripts/30-monitor.sh --load test 120  # 2 minutes of traffic on test, then a snapshot
./scripts/30-monitor.sh --test-alert     # test e-mail through the prod action group
./scripts/30-monitor.sh --dora           # deployment frequency, lead time, change
failure rate
```

In the Azure portal, open:

- appi-eurc-prod-cc-001 → Application map — App Service calling SQL, Blob and Key Vault
- appi-eurc-prod-cc-001 → Availability — /health from three test locations
- Monitor → Alerts — the slot-swap alert from Phase 8
- rg-eurc-prod-cc → app → Deployment slots — production and staging with their versions

RECORD YOUR RESULT

Snapshot taken	
Test alert e-mail received	
DORA output	

Phase 12 — Team exercises

These exercises show the DevOps practices in action. Each team member works on their own computer or in GitHub Codespaces:

```
git clone https://github.com/<owner>/eurc-platform && cd eurc-platform
(cd app && npm ci) && (cd web && npm ci) && (cd tests && npm ci)
cd app && npm run dev     # terminal 1: API with demo store "demo" / manager@demo.test / Password123
cd web && npm run dev     # terminal 2: open http://localhost:5173
```

Exercise 1 — Developer 2 ships a feature, Developer 1 reviews

WHO Developer 2 (author) · Developer 1 (reviewer)

Goal: Add a barcode column to products through a reviewed pull request.

```
git checkout -b feature/pos-1-barcode
```

1. In app/src/migrations.js add migration version 4: IF COL_LENGTH('dbo.products', 'barcode') IS NULL ALTER TABLE dbo.products ADD barcode NVARCHAR(32) NULL;
2. Set "version" to 2.1.0 in app/package.json and web/package.json; run npm test in app/ and web/.

```
git commit -am "feat: product barcode column (expand step)"
git push -u origin feature/pos-1-barcode
gh pr create --fill
```

3. Developer 1 reviews and approves the pull request; then squash-merge.

EXPECTED RESULT CI shows app, web, infra, scripts green; the merge needs Developer 1's approval. CD releases 2.1.0+N; /health shows schemaVersion 4 in every environment.

RECORD YOUR RESULT

Result / PR or run link	

Exercise 2 — The tester adds an automated check

WHO Tester

Goal: Extend the browser tests; only the tester can approve changes to tests/.

```
git checkout -b test/e2e-refund-receipt
```

1. In tests/e2e/pos.spec.mjs, after the refund, assert that the receipt shows REFUNDED.

2. Push, open a pull request. Ask a developer to approve: GitHub still requires the tester's code-owner approval.

EXPECTED RESULT A developer's approval alone does not unlock the merge for tests/. After the merge, the test stage runs the new check.

RECORD YOUR RESULT

Result / PR or run link	

Exercise 3 — CI blocks a broken change

WHO Developer 1

Goal: See the quality gate stop a defect before it is merged.

1. On a new branch change the expected total in web/src/components/Cart.test.jsx from $8.49 to $9.49.
2. Push and open a pull request.
3. Close the pull request without merging.

EXPECTED RESULT The web check fails; the merge button stays disabled; nothing is deployed.

RECORD YOUR RESULT

Result / PR or run link	

Exercise 4 — A bad release is stopped in test

WHO Developer 2 (author) · Developer 1 (reviewer) · Tester

Goal: Show why the tester owns the test suites.

1. In app/src/domain/pricing.js change Math.round to Math.floor in roundHalfUp, and update the expected values in app/test/domain.test.js so CI stays green.
2. Developer 1 approves and merges.
3. Watch CD: dev passes; in test the integration suite expects 849 cents and Playwright expects $8.49.
4. Revert with a new pull request.

EXPECTED RESULT The test job fails; production is never asked for approval and keeps the previous version.

RECORD YOUR RESULT

Result / PR or run link	

Exercise 5 — QA sign-off and rollback

WHO Tester · Lead

Goal: Practise the release decision and the emergency rollback.

1. On the next release the tester approves prod after checking test.
2. Lead: Actions → Rollback production → Run workflow → reason "rollback drill". Tester approves.
3. Run your next release to move forward again.

EXPECTED RESULT The previous version is live within about 1–2 minutes; a slot-swap alert e-mail arrives.

RECORD YOUR RESULT

Result / PR or run link	

Exercise 6 — Operate it

WHO Lead

Goal: Use the operations tools and read delivery metrics.

```
./scripts/30-monitor.sh --load test 120
./scripts/30-monitor.sh --dora
```

1. Open Application Insights → Application map and Failures; find the release annotations on the charts.

EXPECTED RESULT Snapshot lists each environment and slot with version and health; DORA shows deployment frequency, lead time and change failure rate.

RECORD YOUR RESULT

Result / PR or run link	

Phase 13 — Collect evidence

Take these screenshots (or exports) for your submission and tick them off:

#	Evidence	Where	✔
1	CD run graph: build → dev → test → prod → release tag	GitHub → Actions → CD run	☐
2	Production approval dialog with the tester's comment	CD run → Review deployments	☐
3	Run summary with test tables and what-if	CD run → Summary	☐
4	Playwright HTML report	Artifact reports-test	☐
5	Pull request with 4 checks and the CODEOWNERS review	GitHub → Pull requests	☐
6	Environments page (dev, test, prod rules)	Settings → Environments	☐
7	Releases list	GitHub → Releases	☐
8	POS on dev / test / prod (badges, same version)	Each environment URL	☐
9	Register, payment, receipt, daily report	Production URL	☐
10	rg-eurc-prod-cc resources, staging slot, private endpoints	Azure portal	☐
11	Application map, Availability, Alerts	Azure portal	☐
12	20-test-env.sh totals and 30-monitor.sh snapshot	Cloud Shell	☐

Save the files from Cloud Shell before the session ends:

```
zip -r eurc-p02-evidence.zip reports logs
download eurc-p02-evidence.zip
```

Phase 14 — Tear down

Goal	Delete every Azure resource so the lab stops using credit.
Who	Lead
Time	15–20 minutes
Changes	Deletes all eURC Project 02 Azure resources (cannot be undone)

STOP Teardown permanently deletes all stores, sales and receipts. Collect your evidence (Phase 13) first.

```
tmux new -s teardown
./scripts/99-teardown.sh            # add --github to remove GitHub environments and
variables too
```

Type eURC when asked. The script removes the delete locks, the four resource groups, purges the soft-deleted Key Vaults, deletes the budget and verifies.

EXPECTED RESULT "[OK] no eURC Project 02 resource groups remain" and "Teardown complete". The GitHub repository and its history remain for your portfolio.

Check Cost Management the next day; billing data can lag up to 24 hours.

RECORD YOUR RESULT

Teardown completed (date / time)	
Cost Management checked	

Appendix A — Troubleshooting

Symptom	Cause and fix
AADSTS700213: No matching federated identity record found	GITHUB_OWNER / GITHUB_REPO in config/eurc.env must match the repository exactly (case matters). Fix and re-run 01-bootstrap-azure.sh.
AuthorizationFailed … roleAssignments/write on the first deploy	New role assignments take a few minutes. Re-run the failed job; if it persists re-run 01-bootstrap-azure.sh.
Quota error for Standard / S1	Use the B1 fallback in Phase 4.
RequestDisallowedByPolicy	The subscription policy excludes the region. Run preflight, change LOCATION and LOC_SHORT and locationShort in infra/main.bicep.
Smoke test waits with HTTP 503 Starting	The app waits for the database. Portal → web app → Log stream. Login failed for user <token-identified principal> → re-run CD with force_infra.
git push refused: workflow scope	gh auth refresh -s workflow, then git push.
Prod job never starts / no Review deployments button	Approval pending (orange clock). If no reviewer is configured, re-run 02-setup-github.sh after the tester accepted.
A collaborator cannot approve a pull request	They have not accepted the invitation, are the author, or are not a code owner of the changed folder (see CODEOWNERS).
Playwright: Executable does not exist	Re-run the job (the step installs Chromium). Locally: npx playwright install chromium in tests/.
20-test-env.sh shows Key Vault 401 instead of 403 in prod	Public access still enabled; re-run CD with force_infra and check the deployment errors.

Appendix B — Resource names

<sfx> is a stable 5-character suffix derived from the subscription and resource group.

Resource	dev / test	prod
Resource group	rg-eurc-dev-cc · rg-eurc-test-cc	rg-eurc-prod-cc
App Service plan	asp-eurc-<env>-cc-001 (B1)	asp-eurc-prod-cc-001 (S1)
Web app	app-eurc-web-<env>-cc-<sfx>	app-eurc-web-prod-cc-<sfx> + slot staging
App identity	id-eurc-app-<env>-cc	id-eurc-app-prod-cc
SQL server / database	sql-eurc-<env>-cc-<sfx> / sqldb-eurc-core-<env>	sql-eurc-prod-cc-<sfx> / sqldb-eurc-core-prod
Storage / container	steurc<env>cc<sfx> / pos-receipts	steurcprodcc<sfx> / pos-receipts (ZRS)
Key Vault	kv-eurc-<env>-cc-<sfx>	kv-eurc-prod-cc-<sfx>
Monitoring	log-eurc-<env>-cc-001 · appi-eurc-<env>-cc-001	same + ag-eurc-ops-prod, alerts, webtest
Network	—	vnet-eurc-prod-cc-001, snet-eurc-appint-prod-cc, snet-eurc-pe-prod-cc, pe-eurc-sql/blob/kv-prod-cc
Deployment identities	id-eurc-gh-dev-cc · id-eurc-gh-test-cc (rg-eurc-cicd-cc)	id-eurc-gh-prod-cc

Appendix C — Lab completion sign-off

Phase	Outcome (pass / issue)	Date	Initials
0 Prerequisites			
1 Cloud Shell			
2 Configuration			
3 GitHub sign-in			
4 Preflight			
5 Azure bootstrap			
6 GitHub setup			
7 First run (dev, test)			
8 QA sign-off and production			
9 Verification			
10 Hardening			
11 Monitoring			
12 Team exercises			
13 Evidence			
14 Teardown			

Role	Name	Signature	Date
Lead (Developer 1)			
Developer 2			
Tester (QA)			